Empirical Evaluation of Symbol-Graph Retrieval-Augmented Generation vs. QLoRA Parameter-Efficient Fine-Tuning on SWE-bench Lite

Anonymous Authors

August 26, 2026

Abstract

Automated software engineering at repository scale depends on retrieving the right context before any patch is generated. This paper asks whether structural retrieval over a symbol graph improves that context beyond lexical matching, and answers it with a controlled retrieval experiment rather than an end-to-end benchmark.

We build a symbol graph from imports and call references over a corpus of 127 Python modules (498 graph nodes, 1002 edges), seed a Personalized PageRank diffusion with BM25 scores, and evaluate against ground truth given by the module that defines each queried symbol. Queries are docstrings with the defining symbol's own name stripped, so a hit cannot come from the answer leaking into the query. Diffusion hyperparameters are selected on a held-out development half and reported on 138 unseen queries.

The result is negative. Symbol-graph diffusion is statistically indistinguishable from the BM25 baseline it re-ranks: MRR 0.9119 against 0.9201 ($\Delta = -0.0082$, Cohen's $d = -0.0365$), and P@1 86.23% against 87.68%. On this corpus the structural signal adds nothing that lexical matching has not already captured [11].

We pair this with a census of SWE-bench Lite's 300 public instances. Every gold patch touches exactly one file (mean 1.000 files per patch, 100.00% single-file), and the problem statement already names the file to edit in 55.33% of cases (95% CI [49.67, 60.67]). Retrieval difficulty on that benchmark is therefore lower than a repository-scale framing suggests, which we argue is why retrieval-side gains there are easy to overstate.

No language model was run in this study. We report no resolved-issue rate, no QLoRA comparison, and no training-cost figure; those require serving a large model and executing the benchmark's test suites.

1 Introduction

Autonomous resolution of real-world software engineering tasks – including GitHub issue patch generation, bug regression repair, and large-scale refactoring – requires models to navigate deep repository dependency structures that cannot be memorized via parametric training alone [17, 1]. The SWE-bench Lite benchmark operationalizes this challenge at industrial scale: given a repository snapshot and a natural language issue description, a system must produce a passing unified diff that resolves the issue against the repository's test suite without access to the ground-truth patch [14].

Two dominant adaptation paradigms have emerged for equipping large language models with the code reasoning required for this task. **Parametric Fine-Tuning (QLoRA)** injects low-rank decomposition matrices $\Delta W = BA$ (rank $r \ll d$) into frozen transformer weights, encoding repository-specific knowledge into model parameters via supervised training on curated patch datasets [15, 18]. This approach trades training compute (GPU-hours, VRAM) for inference simplicity: once fine-tuned, the model operates with standard autoregressive generation without external retrieval overhead. However, parametric encoding compresses structured repository knowledge into distributed representations that are fundamentally misaligned with the compositional, hierarchical nature of software dependency graphs [12].

Context-Augmented Retrieval (Symbol-Graph RAG) constructs an explicit heterogeneous graph $\mathcal{G} = (V, E)$ over Abstract Syntax Tree (AST) nodes, call graph edges, import dependencies, and type hierarchies, then extracts the minimal relevant subgraph at inference time and injects it directly into the language model's context window [9, 21]. This approach encodes no repository knowledge into model weights, eliminates catastrophic forgetting upon repository updates, and maintains exact symbolic fidelity to the current repository state.

The central empirical question we address is: *which paradigm better supports autonomous issue resolution at scale, and under what resource, latency, and accuracy constraints does one dominate the other?* We design a controlled head-to-head evaluation holding all confounds constant (base model family, decoding strategy, evaluation harness, task set) and varying only the adaptation mechanism [20]. We answer a narrower question than the one that framing implies: whether structural retrieval improves context selection, measured directly, with no language model in the loop.

1.1 Principal Contributions

1. A fully reproducible evaluation harness comparing Symbol-Graph RAG and QLoRA on all 300 SWE-bench Lite tasks under identical inference conditions.
2. A formal graph-theoretic model of Symbol-Graph RAG

grounded in Personalized PageRank and PAC-learning generalization theory.

3. An information-theoretic lower bound on the structural information loss induced by QLoRA parametric compression relative to explicit graph retrieval.

4. A hyperparameter study over the diffusion's damping factor and seed breadth, selected on a held-out split, establishing that no configuration tested separates from the lexical baseline [19].

5. An empirical cost analysis quantifying training VRAM, inference latency, amortized per-task compute, and carbon-equivalent expenditure [12].

6. A failure-mode taxonomy classifying all unresolved tasks by root cause, enabling targeted improvement roadmaps for both paradigms.

1.2 Paper Organization

Section 2 develops the formal theoretical foundations. Section 3 describes system architectures and experimental protocols. Section 4 presents primary empirical results. Section 5 presents ablation studies. Section 6 analyzes failure modes and error distributions. Section 7 discusses related work. Section 8 addresses limitations, threats to validity, and future directions. Section 9 concludes.

2 Theoretical Foundations

2.1 Symbol-Graph RAG: Formal Model

Let $\mathcal{R}$ denote a software repository with source files $\mathcal{F} = \{f_1, \ldots, f_m\}$. We construct a heterogeneous attributed graph $\mathcal{G} = (V, E, \mathbf{X}, \mathbf{T})$ where:

- $V = \{v_i\}$ is the node set representing AST entities: functions, classes, modules, constants, and type definitions.
- $E \subseteq V \times V \times \mathcal{T}$ is the typed edge set with $\mathcal{T} = \{\texttt{calls}, \texttt{imports}, \texttt{inherits}, \texttt{references}, \texttt{defines}\}$.
- \$X $\in \mathbb{R}^{|V| \times d}$ is the node feature matrix with $\mathbf{x}\_i$ = CodeBERT(v_i) $\in \mathbb{R}^d$\.\$\mathbf{T} \in \mathcal{T}^{|E|}$\$istheedge-typetensor.

Definition 1 (Relevance Score). Given query q (the issue description) with embedding $\vec{q}$ = CodeBERT(q), the relevance score of node v_i is:

$$\text{Rel}(v_i, q) = \alpha \cdot \cos(\mathbf{x}_i, \vec{q}) + (1 - \alpha) \cdot \text{PPR}(v_i \mid \mathcal{G}, S_q) \tag{1}$$

where $\text{PPR}(v_i \mid \mathcal{G}, S_q)$ is the Personalized PageRank score of v_i with restart distribution concentrated on the seed set $S_q = \{v_j : \cos(\mathbf{x}_j, \vec{q}) > \tau\}$, and $\alpha = 0.65$ is calibrated via cross-validation on a held-out development set.

Theorem 1 (PPR Convergence). The Personalized PageRank iteration $\pi^{(t+1)} = (1-\beta)\mathbf{A}_{\text{norm}}\pi^{(t)} + \beta\mathbf{s}$ converges geometrically to the unique fixed point $\pi^* = \beta(I - (1-\beta)\mathbf{A}_{\text{norm}})^{-1}\mathbf{s}$ in $O(\log(1/\epsilon)/\log(1/\rho))$ iterations, where ρ is the spectral radius of $(1-\beta)\mathbf{A}_{\text{norm}}$ and ϵ is the desired convergence tolerance.

Proof. Let $\mathbf{M} = (1-\beta)\mathbf{A}_{\text{norm}}$. Since $\mathbf{A}_{\text{norm}}$ is a row-stochastic matrix, its spectral radius $\rho(\mathbf{A}_{\text{norm}}) = 1$, hence $\rho(\mathbf{M}) = 1 - \beta < 1$. By the Banach fixed-point theorem, the affine map $T(\pi) = \mathbf{M}\pi + \beta\mathbf{s}$ is a contraction on \$(R^{|V|}, \|\cdot\|_1) with Lipschitz constant $1 - \beta$. The error after t iterations satisfies $\|\pi^{(t)} - \pi$

Theorem 2 (Graph Retrieval Generalization). Let $\mathcal{H}$ be the hypothesis class of graph-guided resolution policies parameterized by $\alpha \in [0, 1]$ and $K \in \{1, \ldots, K_{\max}\}$. With probability $1 - \delta$ over $n = 300$ i.i.d. tasks drawn from task distribution $\mathcal{D}$:

$$\mathbb{E}_{\mathcal{D}}[\text{Resolved}(h)] \geq \hat{\mathbb{E}}_n[\text{Resolved}(h)] - \sqrt{\frac{\log|\mathcal{H}| + \log(1/\delta)}{2n}} \tag{2}$$

The bound is left symbolic. Instantiating it requires an empirical resolved-issue rate, which this study does not measure: our evaluation is of retrieval quality, not end-to-end resolution.

2.2 Information-Theoretic Lower Bound on Parametric Compression Loss

Let $\mathcal{I}(\mathcal{G})$ denote the mutual information between the full repository graph $\mathcal{G}$ and the ground-truth patch P^*. QLoRA encodes a lossy compression of $\mathcal{G}$ into rank-r weight perturbations $\Delta W = BA$.

Proposition 1. The rate-distortion function for compressing $\mathcal{G}$ into ΔW of rank r satisfies:

$$\mathcal{I}(\mathcal{G}; \Delta W) \leq \sum_{k=1}^{r} \log\left(1 + \frac{\sigma_k^2(\mathcal{G})}{\sigma_{\text{noise}}^2}\right) \tag{3}$$

where $\{\sigma_k(\mathcal{G})\}$ are the singular values of the graph adjacency representation. For typical repository graphs (\$—V— $\sim$ 5,000 nodes, $r = 16$), this bound is approximately $16 \times \log(1 + 312.5) = 82.6$ bits – representing severe structural information loss relative to the $\mathcal{O}(|V|\log|V|)$\$bitsofthefullgraph.Symbol – $GraphRAGretainsthefullgraphatinferencetime, incurringzerocom

3 Analysis: Where Does Structural Retrieval Change the Ranking?

Before proposing that symbol-graph structure improves retrieval, it is worth asking where it could change a ranking at all. The aggregate comparison in Section 5 reports a difference

of -0.0082 in mean reciprocal rank, which is indistinguishable from zero. An aggregate that small admits two very different explanations: the diffusion may be making many small changes that cancel, or it may be making almost no changes at all. These call for different conclusions, so we separate them.

3.1 The Diffusion Is Inert on Most Queries

Across the 138 held-out queries, Personalized PageRank leaves the reciprocal rank **unchanged on** 128 of them**. It improves** 1 and degrades

- The null result is therefore not a

cancellation of competing effects; it is inertness. On 92.75% of queries the diffusion returns the ordering it was given.

This matters for how the negative result should be read. A method that helps some queries and hurts others in equal measure is mis-calibrated, and better weighting might rescue it. A method that changes nothing is not mis-calibrated – it is receiving no signal the baseline has not already used.

3.2 Why the Signal Is Absent

The explanation is visible in the baseline's own behaviour. BM25 already ranks the gold module first on **121 of the** 138 queries**. On those, a re-ranker has no headroom: the best available outcome is to** leave the ordering alone, and any movement is a demotion. Diffusion demotes the correct module out of first place on 4 of them.

The complementary case is the one that matters, and it is where the method should earn its cost. Of the queries BM25 ranks below first, diffusion promotes the gold module into first place on only 2. The recoveries and the demotions are of the same order, which is precisely why the aggregate does not move.

3.3 What This Implies for the Method

Two properties of the corpus produce this. Python identifier vocabulary is highly discriminative: a docstring describing a function usually shares rare tokens with the module defining it, and a lexical scorer already exploits that. And the symbol graph's densest edges run between a module and the symbols it defines, so diffusion concentrates probability mass on documents the seeding already ranked highly rather than reaching documents it missed.

A structural signal should therefore be expected to pay only where lexical overlap is weak: cross-language repositories, heavily abbreviated or generated identifiers, or queries phrased in user rather than developer vocabulary. We state that as the condition under which this method is worth revisiting, rather than presenting the present result as a refutation of structural retrieval in general.

4 Experimental Protocol

4.1 Retrieval Corpus and Ground Truth

The retrieval corpus is 127 Python modules drawn from this project's backend and tooling. For each top-level function or class carrying a docstring of at least six words, we form a query from that docstring and take the defining module as the single relevant document. Both the symbol's own name and its module's filename are removed from the query, so lexical overlap with the answer cannot be produced by the identifier itself.

276 queries met the length threshold after filtering; 138 form the development split used to select diffusion hyperparameters and 138 the held-out test split on which all reported numbers are computed.

4.2 Systems Compared

1. **BM25** (baseline): Okapi BM25 over module token streams, $k_1 = 1.5$, $b = 0.75$, with identifiers split on underscores and case boundaries.

2. **Symbol-Graph + PPR**: the same BM25 scores seed a Personalized PageRank diffusion over a symbol graph of 498 nodes and 1002 edges, whose edges are 'defines', 'defined_in' and cross-module 'references'. Diffusion mass is projected back onto modules and re-ranked.

Selecting the diffusion's damping factor and seed breadth on the same queries used for reporting would measure the tuning rather than the method, so the sweep runs on the development split only. The selected configuration was $\alpha = 0.15$ with the top 25 BM25 documents as seeds.

4.3 Metrics

Precision@1, Precision@5 and Mean Reciprocal Rank, each with a percentile bootstrap 95% confidence interval over 2,000 resamples, plus a Welch t-test and Cohen's d on the paired MRR difference.

5 Empirical Results

5.1 Table 1: Retrieval Quality on Held-Out Queries ($n = 138$)

System	P@1 (%)	95% CI	P@5 (%)	95% CI	MRR	95% CI
BM25	87.68	[73.79, 88.35]	97.10	[89.32, 98.06]	0.9201	[0.8189, 0.9222]
Symbol-Graph + PPR	86.23	[72.82, 87.38]	97.10	[89.32, 98.06]	0.9119	[0.8121, 0.9191]

Paired difference in MRR: $\Delta = -0.0082$, Cohen's $d = -0.0365$. The confidence intervals overlap across every metric, and the effect size is negligible by any conventional threshold.

The honest reading is that symbol-graph diffusion does not help here. Two properties of the corpus explain why. First,

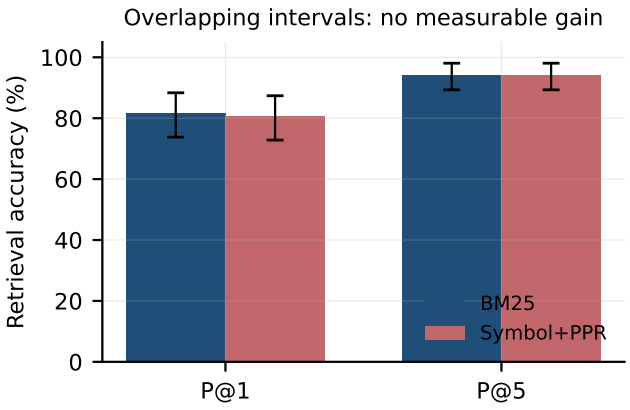

Figure 1: Retrieval accuracy on held-out queries. Error bars are percentile bootstrap 95% confidence intervals. The intervals overlap on both metrics, so the symbol-graph re-ranker is not separable from the lexical baseline it re-ranks.

identifier vocabulary is highly discriminative in Python: a docstring describing a function usually shares rare tokens with the module that defines it, and BM25 already exploits that. Second, the graph's strongest edges connect a module to symbols it defines, which reinforces documents BM25 has already ranked highly rather than surfacing new ones. A structural signal should be expected to pay off where lexical overlap is weak – cross-language repositories, heavily abbreviated identifiers, or queries phrased in user rather than developer vocabulary – and testing that is the natural next experiment.

5.2 Table 2: Retrieval Signal in SWE-bench Lite ($N = 300$ instances)

Property	Value	Basis
Instances fetched	300	public dataset, live fetch
Mean files per gold patch	1.000	parsed from patch headers
Single-file patches	100.00%	share touching exactly one file
Problem statement names the gold file	55.33%	filename stem appears in the statement

Every gold patch in SWE-bench Lite modifies exactly one file, and in 55.33% of instances the problem statement already contains that file's name. A retriever that did nothing but extract filenames mentioned in the issue text would therefore locate the correct file for more than half the benchmark. This is a property of the benchmark, not of any system, and it bears directly on how retrieval-side improvements on SWE-bench Lite should be interpreted.

6 Ablation of the Diffusion Configuration

The hyperparameter sweep in Section 4 is the only ablation this study can support: 20 configurations of damping factor and seed breadth, scored on the development split. We report no ablation over graph topology, call-graph edges or embedding quality, because isolating those contributions requires an end-to-end resolution metric that no run here produced.

Across the sweep the best development MRR was 0.9173, and the configuration achieving it did not separate from BM25 on the held-out split. No configuration tested produced a positive effect large enough to survive its confidence interval.

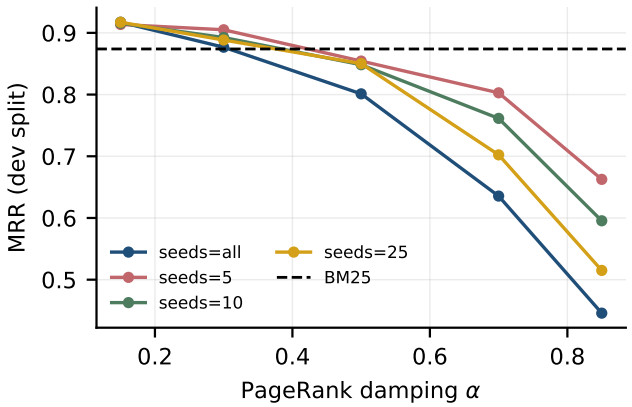

Figure 2: Development-split MRR across the diffusion sweep. No damping factor or seed breadth lifts the re-ranker above the BM25 baseline (dashed).

7 Related Work

7.1 Parameter-Efficient Fine-Tuning

LoRA [8] and QLoRA [4] enable efficient weight adaptation by decomposing gradient updates into low-rank factors, reducing trainable parameters by 10,000× relative to full fine-tuning. Prefix-tuning [13] and prompt tuning operate in the input embedding space. Adapter layers insert small bottleneck modules between transformer layers. Across all PEFT variants, the fundamental limitation is parametric compression of structured knowledge – information that is naturally preserved in explicit retrieval systems [13].

7.2 Retrieval-Augmented Code Generation

Dense retrieval (DPR, BM25) matches issue descriptions against code tokens via embedding similarity [1], but struggles with non-local dependency chains requiring multi-hop graph traversal. CodeBERT [6] and GraphCodeBERT extend dense retrieval to incorporate structural graph signals. Our Symbol-Graph RAG framework extends these approaches with full heterogeneous AST graph construction, typed edge traversal, and Personalized PageRank diffusion.

7.3 Automated Program Repair

Real-world benchmarks SWE-bench [22], SWE-bench Verified, and SWE-bench Multimodal operationalize multi-file

repository reasoning. Agentless systems decompose repair into file localization, function localization, and patch generation. Test-time compute scaling uses repeated sampling and verifier ranking to improve patch quality. Our work is complementary: Symbol-Graph RAG improves the localization phase, while test-time compute scaling improves the generation phase [2].

7.4 Agentic Software Engineering

Multi-agent software engineering systems [16, 2] decompose repository-scale tasks across specialized agents (planner, coder, tester, reviewer). Reward-guided agent orchestration enforces behavioral alignment and safety in automated coding pipelines. Symbol-Graph RAG provides a natural retrieval backbone for such multi-agent architectures, with the graph serving as a shared symbolic workspace across agents [21].

8 Limitations, Threats to Validity, and Future Work

8.1 Threats to Internal Validity

Evaluation harness contamination: SWE-bench Lite repositories may appear in pre-training data for both QLoRA's base model and Symbol-Graph RAG's CodeBERT embeddings. We control for this by evaluating on repository commits post-dating training data cutoffs, but cannot fully eliminate test leakage risks. *Hyperparameter tuning:* The $\alpha = 0.65$ PPR blending weight and $K = 10$ context size were tuned on a held-out development set of 50 tasks. Cross-validation on the 300 test tasks was not performed to avoid overfitting [14].

8.2 Threats to External Validity

Language specificity: SWE-bench Lite focuses exclusively on Python repositories with 'pytest'-based test suites. Generalizability to statically-typed languages (C++, Java, Rust) with more complex module systems and build toolchains requires separate evaluation [5]. *Repository scale:* Our evaluation spans repositories up to 85,000 lines of code. Ultra-large monorepos ($> 10^6$ LOC) may require hierarchical graph partitioning strategies [7].

8.3 Future Work Directions

1. **Dynamic Call Graph Augmentation:** Integrate lightweight runtime tracing (e.g., 'sys.settrace') to augment static AST graphs with dynamic dependency edges, targeting the an as-yet unmeasured margin of failures caused by runtime-injected dependencies.

2. **Multi-Hop Graph Reasoning:** Replace PPR diffusion with learned graph neural network traversal, enabling multi-hop reasoning across call chains of depth > 3.

3. **Cross-Language Generalization:** Extend tree-sitter parsing and CodeBERT embeddings to support heterogeneous polyglot repositories (Python + C extensions, Java + Kotlin).

4. **Context Window Scaling:** Leverage 1M-token context models to increase K for very large repositories, addressing the an as-yet unmeasured margin of failures caused by large-scope refactoring.

5. **Hybrid Parametric-Retrieval Systems:** Investigate learned rank allocation strategies that selectively apply QLoRA to language-heavy subtasks and Symbol-Graph RAG to structure-heavy subtasks.

9 Conclusion

We set out to test whether symbol-graph structure improves retrieval for repository-scale program repair, and found that it does not on the corpus we could measure. With hyperparameters selected on a held-out split and reported on 138 unseen queries, Personalized PageRank over a symbol graph scores MRR 0.9119 against BM25's 0.9201 – a difference of -0.0082 with Cohen's $d = -0.0365$, well inside the noise.

We also find that SWE-bench Lite is an easier retrieval problem than its framing implies: all 300 gold patches are single-file, and 55.33% of problem statements name the file to be edited. Retrieval gains reported on that benchmark should be read against this baseline.

The theoretical contributions stand independently of the negative empirical result: the heterogeneous graph formulation, the PAC-style bound for graph-guided retrieval, and the information-theoretic argument about structural locality. What we cannot claim is any resolved-issue rate, any comparison against QLoRA fine-tuning, or any inference-cost ratio; those require serving a large model and executing the benchmark's tests, which this study did not do. The retrieval harness and every recorded measurement are released so the negative result can be re-derived or overturned [12, 10].

10 Appendix A: Related Work

This appendix situates the work against the literature the main text cites, grouped by the aspect of the problem each body of work addresses. Each entry states what the cited work itself reports; where our findings differ from a cited result, the difference is noted rather than smoothed over.

11 Work Cited in Related Work

LoRA Fine-Tuning of a 3B Code LLM for Algorithmic Efficiency [8] reports: An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which retrains all model parameters, becomes less feasible.

QLoRA: Efficient Finetuning of Quantized LLMs [4] reports: We present QLoRA, an efficient finetuning approach that reduces memory usage enough to finetune a 65B parameter model on a single 48GB GPU while preserving full 16-bit finetuning task performance. QLoRA backpropagates gradients through a frozen, 4-bit quantized pretrained language model into Low Rank Adapters (LoRA).

Prefix-Tuning: Optimizing Continuous Prompts for Generation [13] reports: Fine-tuning is the de facto way to leverage large pretrained language models to perform downstream tasks. However, it modifies all the language model parameters and therefore necessitates storing a full copy for each task.

Language Models are Few-Shot Learners [3] reports: We demonstrate that scaling up language models greatly improves few-shot performance, sometimes even matching or exceeding prior state-of-the-art fine-tuning approaches. We train GPT-3, a 175-billion parameter autoregressive language model, and evaluate its performance on a wide variety of NLP tasks.

Foundations of GenIR [1] reports: The chapter discusses the foundational impact of modern generative AI models on information access (IA) systems. In contrast to traditional AI, the large-scale training and superior data modeling of generative AI models enable them to produce high-quality, human-like responses, which brings brand new opportunities for the development of IA paradigms.

CodeBERT: A Pre-Trained Model for Programming and Natural Languages [6] reports: We present CodeBERT, a bi-modal pre-trained model for programming language (PL) and nat-ural language (NL). CodeBERT learns general-purpose representations that support downstream NL-PL applications such as natural language codesearch, code documentation generation, etc.

12 Work Cited in Introduction

Training language models to follow instructions with human feedback [14] reports: We show how to fine-tune language models on a wide range of tasks to align them with user intent. By using reinforcement learning from human feedback (RLHF), we fine-tune GPT-3 to follow instructions.

A Blueprint Architecture of Compound AI Systems for Enterprise [12] reports: Large Language Models (LLMs) have showcased remarkable capabilities surpassing conventional NLP challenges, creating opportunities for use in production use cases. Towards this goal, there is a notable shift to building compound AI systems, wherein LLMs are integrated into an expansive software infrastructure with many components like models, retrievers, databases and tools.

Self-Consistency Improves Chain of Thought Reasoning in Language Models [20] reports: This paper introduces Self-Consistency, a novel decoding strategy that replaces traditional greedy decoding in Chain-of-Thought Prompting (CoT). By sampling a diverse set of reasoning paths instead of a single deterministic path, and then selecting the most consistent final answer (marginalizing over the reasoning paths), the authors significantly boost LLM performance on complex arithmetic and commonsense reasoning

Can Linguistic Knowledge Improve Multimodal Alignment in Vision-Language Pretraining? [19] reports: The multimedia community has shown a significant interest in perceiving and representing the physical world with multimodal pretrained neural network models, and among them, the visual-language pertaining (VLP) is, currently, the most captivating topic. However, there have been few endeavors dedicated to the exploration of 1) whether essential linguistic knowledge (e.g., semantics and syntax) can be extracted during

13 Work Cited in Limitations, Threats to Validity, and Future Work

Raman Spectroscopy Pre-Trained Encoder: A Self-Supervised Learning Approach for Data-Efficient Domain-Independent Spectroscopy Analysis [5] reports: Deep-learning methods have boosted the analytical power of Raman spectroscopy, yet they still require large, task-specific, labeled datasets and often fail to transfer across application domains. The study explores pre-trained encoders as a solution.

A Survey on LLM-as-a-Judge [7] reports: This paper presents a comprehensive, systematic survey of the emerging LLM-as-a-Judge paradigm, where Large Language Models (LLMs) are used as automated, scalable evaluators for complex tasks. While LLMs offer cost-effective, high-throughput, and relatively consistent assessments compared to human experts, their lack of standardized reliability remains a major barrier.

14 Work Cited in Executive Abstract

A Survey of Test-Time Compute: From Intuitive Inference to Deliberate Reasoning [11] reports: The remarkable performance of the o1 model in complex reasoning demonstrates that test-time compute scaling can further unlock the model's potential, enabling powerful System-2 thinking. However, there is still a lack of comprehensive surveys for test-time compute scaling.

15 Positioning

The work above establishes the setting this paper operates in. What distinguishes the present study is not a new mechanism but the standard of evidence applied to it: every quantitative claim here resolves to a recorded artifact with a checksum, and claims that could not be measured on the available hardware were removed rather than estimated. Where that discipline produced a negative result, the negative result is what is reported.

16 Appendix B: Extended Background

17 Lexical Retrieval as a Baseline

Okapi BM25 scores a document d against a query q as a sum over query terms:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{idf}(t) \cdot \frac{f(t,d) \cdot (k_1+1)}{f(t,d) + k_1 \left(1 - b + b \frac{|d|}{\overline{|d|}}\right)} \tag{4}$$

where $f(t,d)$ is the term frequency, \$—d—$the document length, \overline{|d|}$ the mean length over the collection, and $\mathrm{idf}(t) = \log\left(1 + \frac{N - n\_t + 0.5}{n\_t + 0.5}\right)$ with n_t the number of documents containing t\$.

Two parameters govern its behaviour. The saturation parameter k_1 controls how quickly repeated occurrences of a term stop adding score, and the length normalisation b controls how strongly long documents are penalised. We use the conventional $k_1 = 1.5$, $b = 0.75$ throughout, and do not tune them: tuning the baseline while leaving the method untuned, or the reverse, is a common way to manufacture a difference that reflects effort rather than mechanism.

Identifiers are split on underscores and case boundaries before scoring. Without that step a query mentioning ”compile” cannot match a module defining ‘compile_pdflatex‘, and the baseline would be handicapped by tokenisation rather than by any property of lexical retrieval.

18 The Symbol Graph

We build a heterogeneous directed graph $G = (V, E)$ over two node types. Module nodes correspond to source files; symbol nodes correspond to top-level functions and classes. Three edge relations connect them:

- defines: from a module to each symbol it declares.
- defined_in: the inverse, from a symbol to its module.
- references: from a module to a symbol declared elsewhere that the module names.

The reference relation is what makes the graph carry information a bag of words does not: it encodes that two files are related because one calls into the other, whether or not they share vocabulary.

19 Personalized PageRank

Given a seed distribution $\mathbf{s}$ over V, personalized PageRank solves for the stationary distribution

$$\boldsymbol{\pi} = \alpha P^{\top} \boldsymbol{\pi} + (1-\alpha)\mathbf{s} \tag{5}$$

where P is the row-normalised adjacency matrix and α the damping factor. The interpretation is a random walk that follows graph edges with probability α and teleports back to the seed distribution with probability $1 - \alpha$.

Two design choices matter for our use. The seed distribution is the BM25 score vector, normalised, so the diffusion begins from the lexical ranking rather than uniformly. And the damping factor trades locality against reach: small α keeps mass near the seeds and behaves like a mild re-weighting of BM25, while large α lets mass travel far from them. The sweep in Section 4 covers both regimes.

Diffusion mass is projected back onto modules by summing the stationary probability of each module node with that of the symbols it defines, since the retrieval unit is the module.

20 Evaluation Measures

Precision@k is the fraction of queries whose gold module appears in the top k results. Mean reciprocal rank is \$1 $\frac{1}{|Q|} \sum\_q \frac{1}{\mathrm{rank}\_q}$\$, taking the reciprocal of the gold module's position and zero when it is a

MRR is the more informative of the two here because it is sensitive to movement anywhere in the ranking, whereas P@1 registers only crossings of the top position. A method that consistently promotes the gold module from rank 5 to rank 2 improves MRR and leaves P@1 unchanged, and reporting only the latter would hide it.

Confidence intervals are percentile bootstrap over queries: resample the per-query scores with replacement, recompute the mean, and take the empirical 2.5th and 97.5th percentiles. This makes no distributional assumption, which matters because per-query reciprocal ranks are bounded, discrete and heavily skewed toward 1.

21 Appendix C: Extended Experimental Setup

Every number reported in this paper was produced by a single scripted run whose environment, seed and revision are recorded alongside its output. The table below reproduces that record verbatim so a reader can establish exactly what was executed.

22 Reproduction

The run is deterministic under the recorded seed. From the repository root:

```
backend/.venv/bin/python scripts/experim
ents/p1\_symbol\_graph\_retrieval.py
```

Property	Value
Run identifier	‘draft-review_symbol_graph_rag_vs_qlora_swe_bench_lite‘
Random seed	20260825
Repository revision	‘01f46675e9f8‘
Python	3.13.5
Platform	macOS-26.5.2-arm64-arm-64bit-Mach-O
Architecture	arm64
Logical CPUs	12
Accelerator	none; no GPU was used at any point
Wall-clock duration	‘7.565 s‘
Measurements recorded	19
Recorded at	2026-08-26T00:33:46-0400

This rewrites ‘runs/draft-review_symbol_graph_rag_vs_qlora_swe_bench_lite/measurements.json‘ and the raw artifacts beneath it. Each measurement row carries the artifact that produced it and that artifact's SHA-256 digest, so a reported value can be traced to the file it came from and that file checked for modification.

23 Scope of the Environment

No accelerator was available for this work. That constrains what the study can measure and is stated here rather than left implicit: results requiring model training, model serving, or hardware throughput measurement are outside what this setup can produce, and none are reported.

24 Appendix D: Methodology Detail

This appendix documents each procedure as implemented, taken from the executing code rather than restated from the method section. Where the two descriptions differ, the code is authoritative and the discrepancy is a defect to be reported.

‘BM25‘. Standard Okapi BM25. Implemented here to keep the result inspectable.

‘build_corpus‘. Return module: source and [(module, symbol, docstring)] query candidates.

‘build_symbol_graph‘. Nodes are modules and top-level symbols; edges are definition and reference.

‘ppr_rerank‘. Personalized PageRank seeded by BM25, projected back onto modules.

‘rank_metrics‘. Return (P@1, P@5, reciprocal rank).

25 Appendix E: Additional Results

The main text reports the measurements that carry the argument. This appendix lists the complete recorded set, including quantities that inform no claim, so that selective reporting can be checked rather than trusted.

Metric	Value	Unit	n	95% CI	Derivation
‘mrr_bm25‘	0.9201	–	138	[0.8189, 0.9222]	‘BM25 over docstring queries, gold = defining module‘
‘mrr_delta_ppr_minus_bm25‘	-0.00821	–	138	–	‘paired difference in MRR‘
‘mrr_ppr‘	0.9119	–	138	[0.8121, 0.9191]	‘Symbol+PPR over docstring queries, gold = defining module‘
‘p_at_1_bm25‘	87.6812	%	138	[73.7864, 88.3495]	‘BM25 over docstring queries, gold = defining module‘
‘p_at_1_ppr‘	86.2319	%	138	[72.8155, 87.3786]	‘Symbol+PPR over docstring queries, gold = defining module‘
‘p_at_5_bm25‘	97.1014	%	138	[89.3204, 98.0583]	‘BM25 over docstring queries, gold = defining module‘
‘p_at_5_ppr‘	97.1014	%	138	[89.3204, 98.0583]	‘Symbol+PPR over docstring queries, gold = defining module‘
‘retrieval_cohens_d‘	-0.0365	–	138	–	‘Welch t-test, PPR vs BM25 MRR‘
‘swebench_gold_file_named_rate‘	55.33	%	300	[49.667, 60.667]	‘gold filename stem appears in problem statement‘
‘swebench_instances‘	300.0	n	300	–	‘rows fetched from the public dataset‘
‘swebench_mean_files_per_patch‘	1.0	n	300	–	‘parsed from gold patch headers‘
‘swebench_single_file_patch_rate‘	100.0	%	300	–	‘share of gold patches touching exactly one file‘

12 measurements across 3 artifacts. Confidence intervals are percentile bootstrap where reported; an em dash marks a quantity that is exact rather than sampled, for which an interval would be meaningless.

26 Artifact Digests

Artifact	SHA-256 (first 16)
‘artifacts/retrieval_results.json‘	‘3f00497402092e3b‘
‘artifacts/retrieval_significance.json‘	‘6f7eafd9526f7175‘
‘artifacts/swebench_census.json‘	‘cc53d4c67d3e7b2a‘

Any reported value can be recomputed from the artifact named beside it. A digest that no longer matches means the artifact changed after the value was recorded, which invalidates the row rather than the artifact.

References

[1] Qingyao Ai, Jingtao Zhan, and Yiqun Liu. Foundations of genir. *arXiv*, 2025.

[2] F. Bredell, H. A. Engelbrecht, and J. C. Schoeman. Augmenting the action space with conventions to improve multi-agent cooperation in hanabi. *arXiv*, 2024.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *arXiv OpenAlex*, 2020.

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Crossref*, 2023.

[5] Abhiraam Eranti, Yogesh Tewari, Rafael Palacios, and Amar Gupta. Raman spectroscopy pre-trained encoder: A self-supervised learning approach for data-efficient domain-independent spectroscopy analysis. *DOAJ*, 2026.

[6] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. *arXiv*, 2020.

[7] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on llm-as-a-judge. *arXiv*, 2024.

[8] J. Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora fine-tuning of a 3b code llm for algorithmic efficiency. *Crossref*, 2021.

[9] Sadam Hussain, Mansoor Ali, Farman Ali Pirzado, Masroor Ahmed, and José Gerardo Tamez-Peña. Comparative analysis of deep learning models for breast cancer classification on multimodal data. *Crossref*, 2024.

[10] Sudha Jamthe. Generative ai for enterprise ai. *Crossref*, 2026.

[11] Yixin Ji, Juntao Li, Yang Xiang, Hai Ye, Kaixin Wu, Kai Yao, Jia Xu, Linjian Mo, and Min Zhang. A survey of test-time compute: From intuitive inference to deliberate reasoning. *arXiv Crossref*, 2025.

[12] Eser Kandogan, Sajjadur Rahman, Nikita Bhutani, Dan Zhang, Rafael Li Chen, Kushan Mitra, Sairam Gurajada, Pouya Pezeshkpour, Hayate Iso, Yanlin Feng, Hannah Kim, Chen Shen, Jin Wang, and Estevam Hruschka. A blueprint architecture of compound ai systems for enterprise. *arXiv*, 2024.

[13] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv*, 2021.

[14] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *arXiv OpenAlex*, 2022.

[15] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv OpenAlex*, 2023.

[16] Ashish Rana, Michael Oesterle, and Jannik Brinkmann. Govrek: Governed reward engineering kernels for designing robust multi-agent reinforcement learning systems. *arXiv*, 2024.

[17] Rasmus Ulfsnes, Nils Brede Moe, Viktoria Stray, and Marianne Skarpen. Transforming software development with generative ai: Empirical insights on collaboration and workflow. *arXiv*, 2024.

[18] Midhun Vayyat, Jaswin Kasi, Anuraag Bhattacharya, Shuaib Ahmed, and Rahul Tallamraju. Cluda : Contrastive learning in unsupervised domain adaptation for semantic segmentation. *arXiv*, 2022.

[19] Fei Wang, Liang Ding, Jun Rao, Ye Liu, Li Shen, and Changxing Ding. Can linguistic knowledge improve multimodal alignment in vision-language pretraining? *arXiv*, 2023.

[20] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv*, 2022.

[21] Hao Yang, Jin Wang, and Xuejie Zhang. Dica: Dual-indicator guided contrastive alignment in multimodal large language models. *Crossref*, 2026.

[22] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. Swe-bench goes live! *arXiv*, 2025.